

1. 规范性问题

描述

- 服务内部的配置项没有遵循最佳实践，存在安全或可靠性等问题

数据采集

- k8s 各类资源的配置

分析方法

查看资源配置文件

- 安全性配置未开启
- 缺失探针
- 缺失资源配额
- 挂载本地绝对路径

2. 违反依赖关系

描述

- 引入专家知识：用户按照最初的架构设计确定不同服务的重要性，定义优先等级。
- 高优先级的服务保证更高的可用性，当其依赖于低优先级服务时，能够保证正常的服务降级。
- 如果违反这一关系，说明优先级定义存在问题。

数据采集

- 预定义的服务优先级
- 服务间的 tracing 链路信息/服务间的调用关系图
- 服务的错误率、响应耗时随时间的变化趋势。

分析方法

- 用户预先为服务划分重要等级（P0/P1/P2 等）
- 通过混沌工程等手段，模拟服务之间的故障情况，并在这一过程中收集各服务的错误率、响应耗时随时间的变化情况。
- 利用皮尔逊相关系数等手段，衡量调用上下游服务之间错误率、响应耗时时序的相关程度。并据此判断两服务依赖关系的强弱。
- 约束条件：
 - 如果 A 强依赖于 B，则 B 的优先级不能低于 A。
 - 如果 B 被所有服务弱依赖，则 B 的优先级应该不大于依赖他的所有服务。
- 识别违反上述约束条件的服务等级，向用户报警。

3. 循环依赖

描述

- 服务之间存在双向或循环的调用。

数据采集

- 服务间的 tracing 链路信息/服务间的调用关系图

分析方法

- 根据一段时间内的 tracing 数据构建服务间的调用关系图。算法探测其中的回环链路。
- 给出回环链路调用链，以及导致回环的调用边

4. 单体服务

描述

- 大量服务接口被集成在单一、庞大的服务程序中。

数据采集

- 服务间的 tracing 链路信息/服务间的调用关系图
- 随时间的资源利用率
 - $\text{Resources_Request} > 5 \times \text{Cluster_Median}$ （资源视角）

分析方法

- 架构视角：构建服务间调用关系图，计算某个服务的入度和出度，如果入度+出度>15-20，则说明该服务需要拆分
- 资源视角：统计一段时间内各服务的资源利用率，计算资源利用率的中位数。如果某个服务的资源大于5倍中位数，说明该服务存在过多功能，需要过多资源，难以调度。

5. 单点

描述

- 如果存在少量（1-2）服务，这些服务停机会导致整个系统无法提供核心功能，则称为单点故障。

数据采集

- 预定义的服务优先级
- 服务间的调用关系图
- 服务的副本数、亲和性、资源保证等级配置

分析方法

- 构建服务间调用关系图，并提取 P0 级服务所在的逻辑服务的子图。
- 如果已经存在孤立的 P0 级服务，则报警。
- 运用 Tarjan 算法等方法，识别“一旦故障，则会导致 P0 级服务之间不连通”的一个或少数几个服务。
- 如果这些服务没有配置可用性保证。如 replica、resource request 等，则报警
- 去掉没有可用性保证的节点，剩下的服务数量 < 3 ，则同样报警

6. 过度拆分

描述

- 系统功能模块拆分过细，导致复杂的部署、额外的通信开销、过长的调用链路。

数据采集

- 不同服务部署时间日志
- 服务的处理时延和请求时延
- 服务间的 tracing 链路信息/服务间的调用关系图

分析方法

- 构建服务间调用关系图，如果某个服务只有一个上游和一个下游服务，或该服务与依赖的服务一同部署。则说明该服务与其他服务紧密耦合，拆分不当。
- 如果一个调用链路中出现了多于 5 个服务，则调用链路过长，复杂性增加，稳定性下降。

7. 不合理的资源利用

描述

- 服务的资源利用率过于低于请求的资源量，导致资源浪费；或过于接近极限的资源量，导致稳定性变差。

数据采集

- 服务随时间的资源利用率
- 服务配置中的资源保证

分析方法

- 如果一个服务被定位高优先级，但是没有设定 request，则不合理
- 如果服务的平均实际资源利用率 $< 0.3 * \text{request}$ 时，说明资源浪费
- 如果服务的短期平均实际利用率 $> 0.7 * \text{limit}$ 时，说明资源接近极限，稳定性变差。

8. 配置漂移

描述

- 人为对配置进行了变更，但并未在较短时间内修改回来。

数据采集

- 配置变更的审计日志

分析方法

- 查看每个配置的变更情况，找到由用户进行的配置变更，并衡量其到目前为止的持续时间。如果大于 $1/n$ 天，则说明出现了未记录的人为配置变更。出现配置漂移的风险。
- 是否为用户变更可以通过查看变更者或配置中的特殊 label 或 annotations 来确定。

9. 丧失可维护性

描述

- 软件的演化过程越来越难以进行。发布过程越来越存在挑战。

数据采集

- 启动/扩容到就绪所用时间
- 版本变更和回滚信息日志
- 协同部署

分析方法

- 如果服务的启动/扩容时间超过了 30 秒，则说明其启动逻辑过于复杂，或依赖过多。
- 如果服务的部署频率变低，或每次变更造成的异常回滚比例较高，则说明服务可维护性降低。
- 如果两个或多个服务总是同时部署，则说明这些服务之间耦合过多。

10. 缺少物理隔离

描述

- 关键服务之间缺少物理隔离，服务副本之间缺少容灾配置。

数据采集

- 预定义的服务优先级
- 服务 pod 所在的节点、AZ 信息

分析方法

- 查看 P0 服务所对应的 pod 的实际节点位置。
- 如果有两个 P0 服务位于同一物理节点上，则关键服务间缺少物理隔离
- 如果同一 P0 服务的不同 pod 只唯一同一个 AZ，则关键服务存在可靠性风险。